Sensitivity Labeling for Meetings — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Sensitivity Labeling for Meetings: Microsoft Purview 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Sensitivity Labeling for Meetings (Microsoft Purview) 5

What this solution is . . . 5

Why this matters . . . 6

How it works . . . 6

The meeting label taxonomy (design used throughout) . . . 6

What you get . . . 7

2. Requirements Guide — Sensitivity Labeling for Meetings (Microsoft Purview) 8

2.1 Sensitivity-label licensing, plus Teams Premium for meeting-option protection . . . 8

2.2 Least-privilege roles across Purview and Teams . . . 8

2.3 Security & Compliance PowerShell + MicrosoftTeams toolchain . . . 8

2.4 Meetings-scope capability available and (where required) enabled . . . 9

2.5 A Teams meeting-policy baseline compatible with the taxonomy . . . 9

2.6 Meeting labels authored (Meetings scope) and published . . . 9

2.7 In-scope meetings, classification intent, and stakeholders . . . 9

2.8 A pilot user ring and change control . . . 10

Checklist . . . 10

Global Requirements — Microsoft Purview Solutions 10

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 11

3. Security & Compliance PowerShell + Graph toolchain . . . 11

4. Data sources & scope in scope . . . 11

5. Compliance, records, privacy & legal stakeholders . . . 12

6. A pilot scope and change control . . . 12

3. Implementation Guide — Sensitivity Labeling for Meetings (Microsoft Purview) 12

Step 0 — Connect (all steps) . . . 13

Step 1 — Assess current state (Assess) . . . 13

Step 2 — Design the meeting label taxonomy (Design) . . . 14

Step 3 — Author and publish the meeting labels (Configure) . . . 14

Step 4 — Pilot on real meetings (Pilot) . . . 14

Step 5 — Validate the pilot ring (Pilot → Validate) . . . 15

Step 6 — Expand to the estate (Expand) . . . 15
5. Testing Guide — Sensitivity Labeling for Meetings (Microsoft Purview) 16
Test 1 — Prerequisites and enablement are in place . . . 16
Test 2 — Discovery inventory and Teams baseline are complete . . . 16
Test 3 — Label create dry-run makes no changes . . . 16
Test 4 — Meeting labels exist with the Meetings scope and are published . . . 16
Test 5 — Publish dry-run makes no changes and is reversible . . . 16
Test 6 — Validation gate passes on the pilot ring . . . 17
Test 7 — Lobby is governed by the label . . . 17
Test 8 — Recording is governed by the label . . . 17
Test 9 — Chat and copy are governed by the label . . . 17
Test 10 — Invite protection and content protection . . . 17
Test 11 — Reporting evidence is produced . . . 17
6. Troubleshooting — Sensitivity Labeling for Meetings (Microsoft Purview) 18
The meeting labels don't appear when a pilot user schedules a Teams meeting . . . 18
The meeting-protection controls (lobby / recording / chat / watermark) don't take effect . . . 18
`IPMEET-Set-MeetingLabel.ps1 -Commit` fails or the label isn't created . . . 18
`IPMEET-Validate.ps1` reports a label as "not detected on the policy" but the portal shows it . . . 19
External attendees are (or aren't) held in the lobby unexpectedly . . . 19
Recording still works on a "prevent recording" tier . . . 19
The coverage report is empty or missing labels . . . 19
7. Best Practices — Sensitivity Labeling for Meetings (Microsoft Purview) 20
8. Deliverables Summary — Sensitivity Labeling for Meetings (Microsoft Purview) 21
Documentation . . . 21
Named artifacts . . . 22
PowerShell scripts . . . 22
What the buyer receives . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — Sensitivity Labeling for Meetings: Microsoft Purview

Organizations protect the document and forget the meeting. Board reviews, M&A calls, HR investigations, security-incident bridges, and privileged legal discussions happen live in Microsoft Teams — in meetings created with whatever defaults the organizer clicked: an open lobby that lets anyone with the link walk in, recording anyone can start, a chat that persists and can be copied out, and an invite that travels unprotected. You cannot claim a confidential meeting is protected when its controls are accidental. This solution closes that gap: it uses **sensitivity labels scoped to Meetings in Microsoft Purview** to make lobby, recording, chat, copy, and content protection a **named property of the meeting** — configured, piloted, validated, and defensible.

This is a complete, do-it-today package written for an administrator applying meeting protection for the first time. It delivers a three-tier meeting label taxonomy end-to-end — from reading the current label and Teams-meeting baseline, through authoring and publishing the labels, to piloting them on real meetings and expanding — with every label, policy, and pilot ring named once and reused everywhere, and every change proven on a pilot ring before the estate is touched. Because the meeting-label surface and the Teams-Premium licensing that gates its protections are **evolving**, the guide cites the method and flags every capability "verify your licensing/tenant" rather than asserting a setting or an entitlement as permanent.

What you get

• A plain-language Solution Overview of meeting labels — lobby, recording, chat, copy, watermark,

end-to-end encryption, and invite protection — how they govern the meeting (not the files in it), and how they complement file/email sensitivity labels.

• A Requirements Guide with verify / implement-if-missing for every prerequisite: sensitivity-label

licensing plus **Teams Premium** for the meeting-option protections, least-privilege Purview and Teams roles, the S&C PowerShell + MicrosoftTeams toolchain, the Meetings-scope capability, a compatible Teams meeting baseline, the authored/published labels, in-scope meeting types with stakeholders, and a pilot ring with change control.

• An Implementation Guide of precise, ordered steps: assess labels and the Teams baseline, design the

taxonomy and map, author the meeting labels with their portal protection settings, publish to a pilot ring, **pilot on real meetings**, validate against the gate, then expand with optional default and mandatory labeling.

• Five PowerShell tools — Discovery, a prerequisite/health helper, a safe Configuration/Remediation tool

(`-WhatIf` by default), Validation, and Reporting — built on Security & Compliance PowerShell and the MicrosoftTeams module, **read-only or `-WhatIf` by default with no credentials in code**.

• Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

Sensitive meetings protected from Purview: lobby, recording, chat, copy, and content protection driven by a named sensitivity label and the invite encrypted — validated on a pilot before you scale.

The deployment model

Assess → Design → Configure → **Pilot (real meetings)** → Validate → Expand — with a pilot gate before the estate, and optional default/mandatory labeling turned on only after validation.

Outcome

The meeting blind spot closed — confidential meetings carry a label that holds the wrong people in the lobby, controls recording, keeps chat in the room, and protects the invite — with the determination of which meeting is sensitive, and any records/legal-hold obligation on a recording, left to the owners who make it.

1. Solution Overview — Sensitivity Labeling for Meetings (Microsoft Purview)

What this solution is

Most organizations protect the **document** and forget the **meeting**. A confidential deal, a board review, an HR investigation, or a security-incident bridge is discussed live in a Microsoft Teams meeting — and that meeting was created with whatever defaults the organizer clicked: an open lobby that lets anyone with the link walk in, recording anyone can start, a chat that persists and can be copied out, and an invite that travels unprotected. That is the gap this solution closes. It uses **sensitivity labels scoped to Meetings** in Microsoft Purview to govern the meeting itself:

- **Lobby** — who bypasses the lobby and who must wait to be admitted (e.g. only people in your organization, or only the organizer and co-organizers) — so the wrong people cannot simply join.
- **Recording** — whether the meeting may be recorded and by whom (e.g. recording prevented, or limited to organizers) — so a sensitive conversation is not captured and re-shared.
- **Chat** — whether meeting chat is on, off, or in-meeting only, and whether chat content may be **copied** to the clipboard — so discussion does not leak out of the room.
- **Copy / content protection** — a **watermark** on shared content and video and **end-to-end encryption **for the highest tier (Teams Premium), plus** encryption of the meeting invite** so the calendar item itself is protected.

Everything is done with the **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 / Teams admin centers**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and the **MicrosoftTeams** PowerShell module — no third-party tooling. Meeting labels are configured, **piloted on a small ring first**, validated against pass/fail gates, and then expanded.

> ***A meeting label governs the meeting, not the files shared in it.*** *A meeting-scoped label sets lobby, recording, chat, copy, and content-protection behavior for the Teams meeting and encrypts its invite. It*

> *does **not** classify or encrypt the documents opened during the call — that is the job of **file/email sensitivity labels** (a separate Purview solution). The two are complementary and are commonly deployed together; this solution delivers the meeting half.*

Why this matters

• **The lobby is the perimeter of a live conversation.** If anyone with the link bypasses the lobby, the meeting is only as private as the least careful forward of its invite. A label makes "only my org bypasses the lobby" a property of the meeting, enforced automatically.

• **Recording is forever.** An ad-hoc recording of a confidential meeting becomes a durable, shareable asset that outlives the conversation. A label can prevent recording — or limit it to organizers — for the meetings that warrant it.

• **Chat leaks after the meeting ends.** Meeting chat persists, and copied chat content travels. Turning chat off (or in-meeting only) and preventing copy for the sensitive tiers keeps the discussion in the room.

• **It is defensible and auditable.** "Every board and incident meeting is labeled Highly Confidential — organizer-only lobby, recording prevented, chat off, invite encrypted" is a statement you can evidence with a report, not a hope.

How it works

1. **Assess.** Read the tenant's current state — which sensitivity labels exist and whether any are scoped to **Meetings**, which label policies publish them, and the **Teams meeting policy baseline** (lobby default, recording allowed, chat mode) that meeting labels operate on top of — all read-only.

1. **Design.** Define a **meeting label taxonomy** (below) mapping each tier to concrete lobby, recording, chat, copy, and content-protection settings, aligned to the classification intent your stakeholders set.

1. **Configure.** Author the meeting labels in the Purview portal with the **Meetings** scope and their protection settings, and publish them to users via a label policy. (The label/policy lifecycle is scriptable; the meeting-protection settings are completed in the portal — see the note below.)

1. **Pilot.** Publish the label policy to a **pilot user group** and have them label real meetings first — never tenant-wide on day one. Lobby, recording, and chat changes are **high-impact** for the attendee experience; prove them on the ring.

1. **Validate.** Confirm each intended meeting label exists with the Meetings scope, is published, and that a labeled meeting actually behaves as the tier defines (external attendee held in lobby, recording button unavailable, chat off). Run the pass/fail gates.

1. **Expand.** Broaden the label policy to all intended users, optionally set a **default** meeting label and **mandatory labeling**, and hand ongoing coverage and reporting to operations.

The meeting label taxonomy (design used throughout)

Three tiers are used consistently across the docs, scripts, and portal. Adjust names and settings to your classification scheme — the **determination** of which meeting warrants which tier belongs to your compliance, security, HR, and legal stakeholders; this solution provides the technical mapping.

Label	Lobby (who bypasses)	Meeting chat	Copy chat	Recording	Content protection (Teams Premium)
`Meeting – General`	Tenant default (people in org & guests)	On	Allowed	Allowed	Encrypt invite (optional)
`Meeting – Confidential`	People in my organization (external wait in lobby)	In-meeting only	Prevented	Organizers & co-organizers only	Encrypt invite
`Meeting – Highly Confidential`	Only organizers & co-organizers	Off (or in-meeting only)	Prevented	Prevented	Watermark + end-to-end encryption + encrypt invite

> ***Licensing / plan note — verify against your own tenant.*** *Applying and encrypting the* ***meeting invite*** *builds on standard sensitivity labeling (Microsoft 365 E3/E5). The* ***meeting-option protections*** *driven by the label — lobby, presenter, recording, chat, copy-prevention,* ***watermark****, and* ***end-to-end encryption*** *— depend on* ***Microsoft Teams Premium*** *and on the meeting-scope capability being available and enabled in your tenant. Teams Premium and the Purview meeting-label surface are* ***evolving*** *— confirm current availability, the exact settings offered, and their licensing in the Purview portal and Microsoft's documentation for *your* tenant before you design. Where Teams Premium is unavailable, scope the labels to the invite-encryption and lobby/chat controls that your plan supports and record the Premium-only controls as deferred.*

What you get

• This **Solution Overview**, a **Requirements Guide** (every prerequisite with verify / implement-if-missing), a precise **Implementation Guide**, a **Testing Guide** with pass/fail gates, a **Troubleshooting Guide**, and field-proven **Best Practices**.

• **Five PowerShell tools** — Discovery, a prerequisite/health helper, a safe Configuration/Remediation tool (`-WhatIf` by default), Validation, and Reporting — built on Security & Compliance PowerShell and the MicrosoftTeams module, **read-only or `-WhatIf` by default and with no credentials in code**.

> ***Boundary.*** *Which meetings are sensitive, what the retention/records obligations of a recording are, and any legal-hold or investigation requirement are* ***determinations owned by your compliance, HR, privacy, and legal functions****. This solution provides the technical configuration and the evidence — not the determination.*

2. Requirements Guide — Sensitivity Labeling for Meetings (Microsoft Purview)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (the six shared requirements: compliance licensing, least-privilege Purview roles, the S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) — those are referenced here, not repeated.

2.1 Sensitivity-label licensing, plus Teams Premium for meeting-option protection

Description: A tenant licensed for **sensitivity labels** (Microsoft 365 E3 includes manual labeling; E5 / E5 Compliance for advanced capabilities), **plus Microsoft Teams Premium** for the meeting-specific protections the label enforces — **who bypasses the lobby, presenter, recording control, chat off / copy-prevention, watermark, and end-to-end encryption**. Encrypting the meeting **invite** builds on the labeling baseline. (Global Requirement 1.) **Why:** The invite-protection and basic label surface work on the labeling baseline; the **meeting-option protections** are gated on **Teams Premium** and on the Meetings-scope capability being available in your tenant. **Verify:** M365 admin center → `Billing` → `Licenses` shows the sensitivity-label SKU and **Teams Premium**; `IPMEET-Check-Prereqs.ps1` reports the label and Teams-Premium signals; confirm the **Meetings** scope is selectable on a label in the Purview portal. **Verify against your own plan. Implement:** Assign/purchase Teams Premium (or the plan that includes it) before configuring the Premium-only controls. Where Premium is unavailable, scope the labels to invite-encryption and the lobby/chat controls your plan supports, and record watermark/E2EE/recording-control as **deferred**.

2.2 Least-privilege roles across Purview and Teams

Description: Information Protection Admin (or **Compliance Administrator**) to author and publish sensitivity labels and label policies in Purview; a **Teams Administrator** to read/confirm the Teams meeting-policy baseline (lobby, recording, chat) that the labels operate on top of. Reserve **Global Administrator** for one-time consent/enablement only. (Global Requirement 2 — least privilege.) **Why:** Meeting labeling spans two surfaces — the label (Purview) and the underlying Teams meeting configuration — each with its own least-privilege role. **Verify:** Purview portal → `Settings` → `Roles & scopes` → `Permissions` shows Information Protection Admin; Teams admin center access confirmed; `Connect-IPPSSession` and `Connect-MicrosoftTeams` succeed. **Implement:** Assign the least-privilege role on each surface under change control; do not use Global Administrator for day-to-day label authoring.

2.3 Security & Compliance PowerShell + MicrosoftTeams toolchain

Description: A workstation with **ExchangeOnlineManagement** (provides `Connect-IPPSSession` for `Get-Label` / `Get-LabelPolicy` / `New-Label` / `New-LabelPolicy`) and the **MicrosoftTeams** module (provides `Connect-MicrosoftTeams` / `Get-CsTeamsMeetingPolicy`), with a modern-auth / MFA admin. (Global Requirement 3.) **Why:** Labels and label policies are read/authored through Security & Compliance PowerShell; the Teams meeting baseline is read through the MicrosoftTeams module; the

provided scripts run on both. **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement, MicrosoftTeams`; the connect cmdlets succeed. `IPMEET-Check-Prereqs.ps1` reports which modules/connections are present. **Implement:** `Install-Module ExchangeOnlineManagement, MicrosoftTeams -Scope CurrentUser`; confirm modern-auth sign-in before running steps.

2.4 Meetings-scope capability available and (where required) enabled

Description: The **Meetings** scope for sensitivity labels must be available in your tenant so a label can be scoped to protect Teams meetings and calendar events. In some tenants this capability and/or the Teams-Premium meeting-protection surface must be **enabled/verified** before the scope and its settings appear. **Why:** A meeting-scoped label cannot be authored — and its lobby/recording/chat settings cannot be set — if the Meetings scope is not offered in the label configuration. **Verify:** In the Purview portal, create/edit a label and confirm **Meetings and calendar events** (or equivalent) is an available scope with the protection settings; `IPMEET-Check-Prereqs.ps1` reports whether any Meetings-scoped label already exists. **Verify the exact enablement path against current Microsoft guidance for your tenant** — the meeting-label surface is evolving. **Implement:** Confirm the capability is available for your licensing/tenant and complete any tenant enablement Microsoft documents before authoring the labels; if unavailable, this is a licensing/plan gap (see 2.1).

2.5 A Teams meeting-policy baseline compatible with the taxonomy

Description: The Teams **meeting policy** baseline (`Get-CsTeamsMeetingPolicy`: lobby / `AutoAdmittedUsers`, recording / `AllowCloudRecording`, chat / `MeetingChatEnabledType`) must be **compatible** with the label taxonomy — a label tightens meeting behavior per the tier; it operates within what the tenant Teams configuration allows. **Why:** Meeting labels and Teams meeting policy interact. Understanding the baseline (e.g. is cloud recording tenant-allowed, what is the default lobby) is required so the label's per-meeting settings are expressible and their effect is understood. **Verify:** `IPMEET-Discover.ps1` reports the Teams meeting-policy baseline; confirm it against the taxonomy with your Teams administrator. **Implement:** Where a baseline conflicts with the intended tier (e.g. lobby default is "Everyone"), agree the baseline with the Teams admin under change control and record it in `Meeting Scope – Map`.

2.6 Meeting labels authored (Meetings scope) and published

Description: The meeting labels (`Meeting – General`, `Meeting – Confidential`, `Meeting – Highly Confidential`) authored in the Purview portal with the **Meetings** scope and their lobby/recording/chat/copy/content-protection settings, published via the `Meeting Labels – Policy` label policy. **Why:** A label must exist and be published to a user before that user can apply it when scheduling a meeting; the policy is what surfaces the labels in Teams/Outlook. **Verify:** `Get-Label` lists the meeting labels; `Get-LabelPolicy` shows the policy; `IPMEET-Check-Prereqs.ps1` confirms Meetings-scoped labels are present. Confirm the labels appear when scheduling a Teams meeting as a policy member. **Implement:** Author the labels and their meeting settings (Implementation Guide Step 3) and publish them to the pilot user group first, then expand.

2.7 In-scope meetings, classification intent, and stakeholders

Description: The **meeting types / audiences in scope** (e.g. board, M&A, HR investigations, security incident bridges, legal privileged calls), the **classification intent** mapping meeting sensitivity to the three

tiers, and engagement with **compliance, security, HR, privacy, and legal** stakeholders (Global Requirements 4 & 5). **Why:** Which meeting gets which label is a business/classification decision, and recording and chat controls have HR/privacy/legal implications — the determination is the stakeholders', the implementation is yours. **Verify:** The in-scope meeting types and the tier mapping are agreed; named approvers recorded. **Implement:** Capture the classification intent and tier mapping before publishing labels; agree that recording/records obligations for labeled meetings are owned by compliance/records.

2.8 A pilot user ring and change control

Description: A **pilot ring** — `Meeting Labels – Pilot Ring – Stage1`: a small set of named pilot users who will label real meetings — plus a **change-control** process (Global Requirement 6). **Why:** Lobby, recording, and chat changes are high-impact for the attendee experience; prove them with a pilot group on real meetings before publishing to the estate. Turning chat off on a busy recurring meeting without piloting generates help-desk tickets. **Verify:** The pilot user ring is defined and a change window/approval path exists. **Implement:** Define the ring and agree change control before publishing labels beyond the pilot.

Checklist

- Sensitivity-label licensing confirmed; **Teams Premium** available for meeting-option protection (or Premium-only controls deferred)
- Information Protection Admin (Purview) + Teams Administrator assigned least-privilege
- ExchangeOnlineManagement + MicrosoftTeams installed; `Connect-IPPSSession` and `Connect-MicrosoftTeams` succeed
- **Meetings** scope available/enabled for labels in the Purview portal (verified for your tenant)
- Teams meeting-policy baseline (lobby / recording / chat) read and compatible with the taxonomy
- Meeting labels authored (Meetings scope) and `Meeting Labels – Policy` published to pilot users
- In-scope meeting types + tier mapping agreed; stakeholders named; records obligations owned
- `Meeting Labels – Pilot Ring – Stage1` (pilot users) defined; change control agreed

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or***

***flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Sensitivity Labeling for Meetings (Microsoft Purview)

Follow these steps in order. The model is **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. Every write is **piloted on the** `Meeting Labels – Pilot Ring – Stage1` **first**, previewed with `-WhatIf`, and reversible. Named objects are introduced once and reused everywhere:

Object	Name
Meeting labels	`Meeting – General`, `Meeting – Confidential`, `Meeting – Highly Confidential`
Label policy	`Meeting Labels – Policy`
Pilot ring (users)	`Meeting Labels – Pilot Ring – Stage1`
Scope/design map	`Meeting Scope – Map`
Coverage report	`Meeting Labels – Coverage Report`

> ***Complete the meeting-protection settings in the portal.*** *The label/policy* ***lifecycle*** *(create, publish, add-to-policy) is scriptable and is what the provided tools automate safely. The* ***meeting-option protection settings themselves*** *— who bypasses the lobby, presenter, recording, chat mode, copy-prevention, watermark, end-to-end encryption — are configured on the label in the* ***Purview portal*** *(they require Teams Premium and depend on the evolving meeting-label surface). Author the label's meeting settings in the portal as described in Step 3; use the scripts for the safe, repeatable lifecycle and for validation/reporting.* ***Verify each setting's availability against your licensing/tenant.***

Step 0 — Connect (all steps)

Open PowerShell 5.1+ or 7+ and connect with a modern-auth/MFA admin:

```
Connect-IPPSSession                 # Security & Compliance PowerShell (labels + policies)
Connect-MicrosoftTeams              # Teams meeting-policy baseline (read-only here)
```

Keep credentials out of scripts — sign in interactively. **Pass/fail:** both connections succeed and `Get-Label` / `Get-CsTeamsMeetingPolicy` return without error.

Step 1 — Assess current state (Assess)

1. Run `IPMEET-Discover.ps1`. Read-only, it reports: every sensitivity label and **whether any is

scoped to Meetings**, the label policies that publish them, and the** Teams meeting-policy baseline** (lobby / `AutoAdmittedUsers`, recording / `AllowCloudRecording`, chat / `MeetingChatEnabledType`).

1. Run `IPMEET-Check-Prereqs.ps1`. It confirms modules/connections, the roles, the Teams-Premium and

Meetings-scope signals, and whether meeting labels already exist — PASS/GAP per signal.

1. Record every GAP against the Requirements checklist and remediate before configuring.

Pass/fail: the discovery inventory and prereq report complete; the Teams baseline is captured in `Meeting Scope – Map`.

Step 2 — Design the meeting label taxonomy (Design)

1. With your compliance/security/HR/legal stakeholders, confirm the three-tier taxonomy from the Solution

Overview and the concrete settings per tier (lobby, chat, copy, recording, content protection).

1. Map your in-scope **meeting types** to tiers (e.g. board & M&A → `Meeting – Highly Confidential`;

internal-confidential project calls → `Meeting – Confidential`; everyday meetings → `Meeting – General`). Record the mapping and the Teams baseline in `Meeting Scope – Map`.

1. Confirm which controls your licensing supports (Teams Premium for recording-control / watermark /

E2EE); mark any unsupported control **deferred** rather than designing around an unavailable feature. **Pass/fail:** `Meeting Scope – Map` is agreed and signed off; the determination of which meeting is which tier is owned by stakeholders, not by IT.

Step 3 — Author and publish the meeting labels (Configure)

Do this **for the pilot first** — publish only to `Meeting Labels – Pilot Ring – Stage1`.

1. **Create the labels (portal).** Purview portal → **Information Protection → Sensitivity labels →

Create a label**. For each tier, set the name/tooltip and choose the scope** Meetings and calendar events** (verify the exact scope wording for your tenant). On the meeting-settings page configure:

- **Lobby** — who can bypass the lobby (`Meeting – Confidential` = people in my organization;

`Meeting – Highly Confidential` = only organizers & co-organizers).

- **Meeting chat** — on / in-meeting only / off, per tier; **prevent copying chat content** for the

Confidential and Highly Confidential tiers.

- **Recording** — allow / limit to organizers / prevent, per tier.

- **Content protection (Teams Premium)** — **watermark** and **end-to-end encryption** for

`Meeting – Highly Confidential`; **encrypt the meeting invite** where the tier calls for it.

You can scaffold the **label object and policy** safely with the script:

```
.\IPMEET-Set-MeetingLabel.ps1 -Action CreateLabels -WhatIf     # preview
.\IPMEET-Set-MeetingLabel.ps1 -Action CreateLabels -Commit     # create the label objects
```

Then complete each label's **meeting-protection settings in the portal** (they are not asserted by the script). Re-read with `Get-Label`.

1. **Publish to the pilot (policy).** Create/point the `Meeting Labels – Policy` at the pilot users:

```
.\IPMEET-Set-MeetingLabel.ps1 -Action PublishPolicy -PolicyMembers "MeetingLabels-Pilot@contoso.com" -W
.\IPMEET-Set-MeetingLabel.ps1 -Action PublishPolicy -PolicyMembers "MeetingLabels-Pilot@contoso.com" -C
```

In the portal, confirm the policy lists the three meeting labels and (optionally, for later) a default label and mandatory labeling — **leave default/mandatory off during the pilot**. **Pass/fail:** `Get-Label` shows the three labels; `Get-LabelPolicy` shows `Meeting Labels – Policy` scoped to the pilot users; the labels appear when a pilot user schedules a Teams meeting and **not** for a non-member.

Step 4 — Pilot on real meetings (Pilot)

1. Have pilot users apply each tier to real (or representative) Teams meetings — a `Meeting – Highly Confidential meeting for a mock board call, a Meeting – Confidential` for a project call.

1. Observe the enforced behavior: an external attendee is held in the lobby; the recording control reflects the tier; chat is off / in-meeting only and cannot be copied; the invite is encrypted; the watermark appears when sharing (Premium). **Pass/fail:** each piloted meeting behaves as its tier defines; discrepancies are recorded for troubleshooting, not worked around.

Step 5 — Validate the pilot ring (Pilot → Validate)

1. Run `IPMEET-Validate.ps1`. Read-only, it confirms each intended meeting label exists **with the Meetings scope**, is** published** in `Meeting Labels – Policy` to the pilot users, and that the Teams baseline is compatible — PASS/FAIL per gate.

1. Have a pilot organizer confirm real behavior end-to-end on a labeled meeting (lobby holds an external attendee; recording unavailable on the Highly Confidential tier; chat off / no copy).

1. Complete the **Testing Guide** on the ring. **Pass/fail:** all Testing-Guide gates pass on `Meeting Labels – Pilot Ring – Stage1`.

Step 6 — Expand to the estate (Expand)

1. Broaden `Meeting Labels – Policy` from the pilot user group to all intended users (portal or `IPMEET-Set-MeetingLabel.ps1 -Action PublishPolicy -PolicyMembers <all-intended> -Commit`). Confirm the labels surface at meeting-creation for the wider audience.

1. Optionally set a **default meeting label** and **mandatory labeling** in the policy — only after the pilot validates and stakeholders approve; communicate the change to users first.

1. Run `IPMEET-Report.ps1 -CsvPath .\MeetingLabelsReport.csv` to produce `Meeting Labels – Coverage Report` — the labels, their scope, and their publication/target coverage — as the management evidence artifact, and re-run it on a schedule to keep coverage healthy. **Pass/fail:** the labels are published to all intended users; the report shows the taxonomy present, scoped to Meetings, and targeted as designed.

Rollback

- **Un-publish** by removing a label from the policy (portal, or `Set-LabelPolicy ... -RemoveLabel`) or narrowing the policy back to the pilot users — labels stop surfacing to users immediately.
- **Retire a label** with `Remove-Label` only after confirming it is applied to no in-flight meetings you depend on; prefer un-publishing over deletion.
- Meeting-protection settings changed on a label take effect on **new** meetings; existing recurring meetings may need re-labeling. Keep `Meeting Scope – Map` as the record of the intended state.

Order of operations

`Assess labels + Teams baseline → design taxonomy + map → author meeting labels (Meetings scope, portal settings) → publish policy to pilot users → pilot on real meetings → validate ring → expand policy (+ optional default/mandatory) → report coverage`

5. Testing Guide — Sensitivity Labeling for Meetings (Microsoft Purview)

Run after each configuration step. Every test is safe (read-only or a `-WhatIf` preview) except where it asks you to observe real behavior on a labeled meeting. Validate on the `Meeting Labels – Pilot Ring – Stage1` before treating the tenant as production-ready.

Test 1 — Prerequisites and enablement are in place

Do: Run `IPMEET-Check-Prereqs.ps1`. **Expected:** ExchangeOnlineManagement + MicrosoftTeams modules present and connected; Information Protection Admin / Teams Admin roles reported; the Teams-Premium and **Meetings-scope** signals reported; any Meetings-scoped labels that already exist are listed. Every gap (e.g. Teams Premium missing → Premium-only controls deferred) is **recorded, not guessed**.

Test 2 — Discovery inventory and Teams baseline are complete

Do: Run `IPMEET-Discover.ps1`; spot-check the Teams meeting policy in the Teams admin center. **Expected:** Every sensitivity label is listed with whether it is **scoped to Meetings**; the label policies publishing them are listed; the Teams meeting-policy baseline (lobby / `AutoAdmittedUsers`, recording / `AllowCloudRecording`, chat / `MeetingChatEnabledType`) is reported and matches the admin center.

Test 3 — Label create dry-run makes no changes

Do: Run `IPMEET-Set-MeetingLabel.ps1 -Action CreateLabels -WhatIf`. **Expected:** It states which labels it *would* create and changes nothing; `Get-Label` afterward shows no new labels until `-Commit`.

Test 4 — Meeting labels exist with the Meetings scope and are published

Do: Run `Get-Label | Format-Table DisplayName, Name, ContentType` and `Get-LabelPolicy`; as a pilot user, start scheduling a new Teams meeting. **Expected:** The three meeting labels exist and carry the **Meetings** scope; `Meeting Labels – Policy` is published to the pilot users; the labels appear in the meeting-scheduling UI for a policy member and **not** for a non-member.

Test 5 — Publish dry-run makes no changes and is reversible

Do: Run `IPMEET-Set-MeetingLabel.ps1 -Action PublishPolicy -PolicyMembers "MeetingLabels-Pilot@contoso.com" -WhatIf`. **Expected:** The dry-run states the

policy/label/membership it *would* set plus the reverse action; a follow-up `Get-LabelPolicy` shows the policy unchanged — `-WhatIf` truly changed nothing.

Test 6 — Validation gate passes on the pilot ring

Do: Run `IPMEET-Validate.ps1`. **Expected:** PASS for each gate — every intended label exists with the Meetings scope, is published in `Meeting Labels – Policy` to the pilot users, and the Teams baseline is compatible with the taxonomy. Any FAIL names the specific missing object.

Test 7 — Lobby is governed by the label

Do: As a pilot organizer, schedule a `Meeting – Highly Confidential` meeting and have an **external** attendee attempt to join. **Expected:** The external attendee is **held in the lobby** (only organizers & co-organizers bypass); on a `Meeting – General` meeting the same attendee is admitted per the tenant default. (Verify the exact lobby behavior against your Teams configuration.)

Test 8 — Recording is governed by the label

Do: Join a `Meeting – Highly Confidential` meeting and attempt to start recording; repeat on a `Meeting – Confidential` meeting as a non-organizer. **Expected:** Recording is **prevented** on the Highly Confidential tier; on the Confidential tier a non-organizer cannot start recording (organizers & co-organizers only). Requires Teams Premium — record as deferred if Premium is not licensed.

Test 9 — Chat and copy are governed by the label

Do: In a `Meeting – Highly Confidential` meeting, check the chat; in a `Meeting – Confidential` meeting, attempt to copy chat content. **Expected:** Chat is **off** (or in-meeting only) on the Highly Confidential tier; copying chat content is **prevented** on the Confidential and Highly Confidential tiers.

Test 10 — Invite protection and content protection

Do: Inspect the calendar invite of a `Meeting – Confidential` meeting; share content in a `Meeting – Highly Confidential` meeting. **Expected:** The invite is **encrypted/protected** per the tier; a **watermark** appears on shared content/video and (where configured) **end-to-end encryption** is active on the Highly Confidential tier (Teams Premium). Record as deferred where Premium is unavailable.

Test 11 — Reporting evidence is produced

Do: Run `IPMEET-Report.ps1 -CsvPath .\MeetingLabelsReport.csv`. **Expected:** `Meeting Labels – Coverage Report` lists each meeting label, its scope, and its publication/target coverage — usable as the management evidence artifact and re-runnable on a schedule.

Pass/fail summary

All gates green on `Meeting Labels – Pilot Ring – Stage1` — labels exist with the Meetings scope, are published to the pilot, lobby/recording/chat/copy/invite behavior matches each tier on a real meeting, and the coverage report is produced — before expanding to the estate. Any deferred

(Premium-gated) control is recorded explicitly, not silently skipped.

6. Troubleshooting — Sensitivity Labeling for Meetings (Microsoft Purview)

Symptoms, likely causes, and fixes for the meeting-label lifecycle. Stay in **simulation/pilot** while diagnosing — reproduce on `Meeting Labels – Pilot Ring – Stage1` before touching the wider estate, and prefer un-publishing a label over deleting it.

The meeting labels don't appear when a pilot user schedules a Teams meeting

- **Not published to that user.** Confirm `Meeting Labels – Policy` targets the pilot users

(`Get-LabelPolicy -Identity "Meeting Labels - Policy"`); add the user with the portal or `IPMEET-Set-MeetingLabel.ps1 -Action PublishPolicy -Commit`.

- **Publication lag.** Label-policy changes can take time to reach clients (commonly up to ~24 h). Have

the user sign out/in of Teams/Outlook and retry before assuming failure.

- **Label not Meetings-scoped.** Run `IPMEET-Validate.ps1`; if a label's `ContentType` doesn't include

the Meetings scope, re-author the scope in the portal (Information Protection → Sensitivity labels → the label → scope **Meetings and calendar events**). Verify the exact scope wording for your tenant.

The meeting-protection controls (lobby / recording / chat / watermark) don't take effect

- **Teams Premium not licensed.** Recording control, watermark, and end-to-end encryption are

Teams Premium features. `IPMEET-Check-Prereqs.ps1` reports the Premium signal — if it's missing, those controls are **deferred**, not broken. Confirm licensing in the M365 admin center.

- **Settings authored on the wrong surface.** The label *lifecycle* is scriptable, but the

meeting-option protection settings themselves are set **on the label in the portal**. If a control is unset there, the script cannot assert it. Re-open the label's meeting-settings page and confirm each control (lobby bypass, chat mode, recording, watermark, E2EE).

- **Applies to new meetings only.** Changing a label's settings affects **new** meetings; existing

recurring meetings may need re-labeling. Re-create or re-label the meeting to test the new setting.

- **Teams meeting policy overrides.** A restrictive/permissive tenant Teams meeting policy can mask the

label behavior. Compare the label intent against the Teams baseline captured in `Meeting Scope – Map` (`Get-CsTeamsMeetingPolicy`); align, don't work around.

`IPMEET-Set-MeetingLabel.ps1 -Commit` fails or the label isn't created

- **Not connected / wrong session.** Ensure `Connect-IPPSSession` succeeded in this session

(`Get-Command Get-Label` resolves). The label cmdlets live in Security & Compliance PowerShell, not the Teams module.

- **Insufficient role.** Creating/publishing labels needs an Information Protection / Compliance

Administrator role. Confirm membership in Purview → **Roles & scopes / Permissions**.

- **Duplicate name.** A label or policy with the same name already exists. Run `IPMEET-Discover.ps1` /

`Get-Label` first; reuse the existing object rather than creating a duplicate.

`IPMEET-Validate.ps1` reports a label as "not detected on the policy" but the portal shows it

- **Module-version property shape.** Label-to-policy attachment is exposed under different property names

across module versions; the validator checks the common shapes. Confirm in the portal (Information Protection → Label policies → the policy → **Labels**) — if it's listed there, treat the validator line as informational and note the module version. Update the ExchangeOnlineManagement module if it is significantly out of date.

External attendees are (or aren't) held in the lobby unexpectedly

- **Lobby is a combined result.** The effective lobby behavior comes from the label's lobby setting **and**

the tenant/organizer Teams meeting policy (`AutoAdmittedUsers`). Verify both; the label tightens, it doesn't override a more permissive test expectation silently. Test with a real external account, not a guest already in the tenant.

Recording still works on a "prevent recording" tier

- **Premium dependency** (see above) — without Teams Premium the recording control is deferred.
- **Cached client.** Have the participant fully restart Teams; policy/label evaluation is cached per

session. Re-test on a freshly created meeting of that tier.

The coverage report is empty or missing labels

- **No Meetings-scoped labels yet.** `IPMEET-Report.ps1` only lists meeting-relevant labels; if none are

authored/scoped, author them (Implementation Step 3) first.

- **CSV not written.** The report is console-only unless you pass `-CsvPath`. Provide a writable path

(e.g. `-CsvPath .\MeetingLabelsReport.csv`) and confirm you have write permission to that folder.

When to stop and escalate

• A required control is unavailable on your licensing/tenant (e.g. Teams Premium) — record it **deferred** and route the licensing decision to the owner; do not design around an unavailable feature.

• **Which meeting belongs to which tier** is a stakeholder determination (compliance/security/HR/legal), not a technical fix — capture the mapping in `Meeting Scope – Map` and defer the determination.

• If behavior can't be reconciled with the label and Teams baseline, keep the policy scoped to the pilot ring, gather `IPMEET-Validate.ps1` + `IPMEET-Report.ps1` output as evidence, and raise with Microsoft support rather than expanding a policy that doesn't behave as designed.

7. Best Practices — Sensitivity Labeling for Meetings (Microsoft Purview)

• **Confirm the meeting-label surface before you design anything.** The **Meetings** scope, its exact protection settings, and the Teams-Premium licensing that gates them are evolving — read them from the Purview label-configuration UI and Microsoft's current docs for *your* tenant rather than assuming a setting or a license entitlement that may have moved.

• **Label the meeting *and* the files — they are different jobs.** A meeting label governs lobby, recording, chat, copy, and the invite; it does **not** classify or encrypt the documents shared in the call. Deploy meeting labels alongside file/email labels so the conversation and its content are both protected, and say so to users so they don't assume one covers the other.

• **Keep the taxonomy small and named.** Three tiers (`General`, `Confidential`, `Highly Confidential`) that users can choose confidently beat ten they'll guess at. Reuse the exact names across the portal, the scripts, and `Meeting Scope – Map`.

• **Pilot on real meetings before you publish widely.** Lobby, recording, and chat changes are felt immediately by every attendee. Prove each tier on the `Meeting Labels – Pilot Ring – Stage1` with real organizers before broadening the policy — never publish tenant-wide on day one.

• **Turn on default and mandatory labeling last, and communicate first.** A default meeting label and mandatory labeling drive coverage, but flipping them without warning surprises organizers mid-schedule. Enable them only after the pilot validates and after you've told users what will change.

• **Right-size the lobby, don't just lock it.** "Only organizers & co-organizers bypass" is correct for a board or incident call and wrong for an all-hands. Match the lobby setting to the meeting type in the taxonomy so protection doesn't become friction users route around.

• **Recording control is a records decision, not just a toggle.** Whether a sensitive meeting may be recorded — and how long a recording is kept or held — is owned by compliance/records/legal. Configure the technical control the label enforces; leave the retention/hold determination to the owners who make it.

• **Mind the interaction with Teams meeting policy.** Meeting labels operate on top of the tenant Teams

meeting configuration (lobby default, whether cloud recording is allowed, chat mode). Read the baseline with `Get-CsTeamsMeetingPolicy`, record it in `Meeting Scope – Map`, and change it only with the Teams admin under change control.

- **Treat watermark and end-to-end encryption as Premium-gated and verify per tenant.** These (and

recording control) depend on Teams Premium. Where Premium is unavailable, scope the labels to the invite/lobby/chat controls your plan supports and record the Premium-only controls as **deferred** — don't design around a feature the tenant can't enforce.

- **Keep automation safe-by-default and secret-free.** Run discovery, validation, and reporting read-only;

gate every write behind `-WhatIf` → `-Commit`; sign in interactively with modern auth. No credentials, tokens, or secrets belong in a script or a document.

- **Re-report on a schedule.** Coverage drifts as new meeting types and organizers appear. Re-run

`IPMEET-Report.ps1` periodically so `Meeting Labels – Coverage Report` stays a live evidence artifact, not a one-time snapshot.

- **Keep determinations with their owners.** The solution configures and evidences meeting protection;

which meeting is sensitive, and any records/legal-hold/privacy obligation on a recording or chat, is a determination for compliance, security, HR, and legal. Keep that separation explicit so technical configuration is never mistaken for a compliance ruling.

8. Deliverables Summary — Sensitivity Labeling for Meetings (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Meeting sensitivity labels — lobby, recording, chat, copy, content protection, and invite encryption; how they govern the meeting (not the files) and complement file/email labels (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: assess labels + Teams baseline, design taxonomy, author meeting labels (Meetings scope + portal settings), publish to pilot, pilot on real meetings, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation

Document	Purpose
7. Best Practices	Field-proven recommendations

Named artifacts

Artifact	Name
General meeting label	`Meeting – General`
Confidential meeting label	`Meeting – Confidential`
Highly Confidential meeting label	`Meeting – Highly Confidential`
Label policy	`Meeting Labels – Policy`
Pilot ring (users)	`Meeting Labels – Pilot Ring – Stage1`
Scope/design map	`Meeting Scope – Map`
Coverage report	`Meeting Labels – Coverage Report`

PowerShell scripts

Script	What it does
`IPMEET-Discover.ps1`	Reports current state: every sensitivity label and whether any is scoped to Meetings, the label policies publishing them, and the Teams meeting-policy baseline (lobby / recording / chat) — Security & Compliance PowerShell + MicrosoftTeams; read-only
`IPMEET-Check-Prereqs.ps1`	Prerequisite/health helper: modules, connections, roles, the Teams-Premium and Meetings-scope signals, and whether meeting labels already exist — PASS/GAP per signal; read-only
`IPMEET-Set-MeetingLabel.ps1`	Configuration/Remediation: creates the meeting label objects and publishes/points the `Meeting Labels – Policy` at the pilot (or expansion) users; **defaults to** `-WhatIf` (prints the plan + reverse action), applies only with `-Commit`; meeting-protection settings are completed in the portal; no secrets in code
`IPMEET-Validate.ps1`	Validation: PASS/FAIL that each intended meeting label exists with the Meetings scope, is published in `Meeting Labels – Policy` to the intended users, and the Teams baseline is compatible — read-only
`IPMEET-Report.ps1`	Coverage report of meeting labels — each label, its scope, and its publication/target coverage — written to CSV as `Meeting Labels – Coverage Report`; read-only

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable method following the model: Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

Sensitive meetings protected from Purview — lobby, recording, chat, copy, and content protection driven by a named sensitivity label and the invite encrypted — proven on a pilot ring before you scale, with the determination of which meeting is sensitive, and any records/legal-hold obligation on a recording, left to the owners who make it.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IPMEET-Check-Prereqs.ps1

Purpose: Confirms the tooling and tenant signals are in place before you configure meeting labels: the required PowerShell modules and connections, the Security & Compliance and Teams roles reachable, whether any Meetings-scoped labels already exist, and best-effort Teams-Premium / Meetings-scope availability signals. Read-only — makes no change. Supports the Assess step and the Testing Guide. HOW TO RUN .\IPMEET-Check-Prereqs.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession and Connect-MicrosoftTeams (modern-auth /MFA admin). No secret is stored in the script. EXPECTED RESULTS A PASS/GAP line per signal: modules present, connections live, Meetings-scoped labels present or absent, Teams meeting-policy readable. Any gap (e.g. Teams Premium not confirmed → Premium-only controls deferred) is reported, not guessed. .NOTES Purview — Sensitivity Labeling for Meetings. Read-only. Verify your licensing/tenant: Teams Premium and the Meetings-scope capability gate the meeting-option protections; confirm both in the portal/ licensing rather than inferring them from a script signal alone. > [CmdletBinding()] param([string]$MeetingScopeMatch = 'Meeting')

How to run: .\IPMEET-Check-Prereqs.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession and Connect-MicrosoftTeams (modern-auth /MFA admin). No secret is stored in the script. EXPECTED RESULTS A PASS/GAP line per signal: modules present, connections live, Meetings-scoped labels present or absent, Teams meeting-policy readable. Any gap (e.g. Teams Premium not confirmed → Premium-only controls deferred) is reported, not guessed. .NOTES Purview — Sensitivity Labeling for Meetings. Read-only. Verify your licensing/tenant: Teams Premium and the Meetings-scope capability gate the meeting-option protections; confirm both in the portal/ licensing rather than inferring them from a script signal alone. > [CmdletBinding()] param([string]$MeetingScopeMatch = 'Meeting')

IPMEET-Discover.ps1

Purpose: Reports current state so you can assess before configuring meeting labels: every sensitivity label and whether any is scoped to Meetings, the label policies that publish them, and the Teams meeting-policy baseline (lobby / AutoAdmittedUsers, recording / AllowCloudRecording, chat / MeetingChatEnabledType) that meeting labels operate on top of. Read-only — it makes no change to any label, policy, or Teams setting. Supports the Assess step. HOW TO RUN .\IPMEET-Discover.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession (Security & Compliance PowerShell, for Get-Label / Get-LabelPolicy) and Connect-MicrosoftTeams (for Get-CsTeamsMeetingPolicy). Sign in interactively with a modern-auth/MFA admin; no secret is stored in the script. EXPECTED RESULTS A summary: labels total and how many are Meetings-scoped (listed), the label policies present, and the Teams meeting-policy baseline — each as a PASS/INFO line to record against the Requirements checklist. .NOTES Purview — Sensitivity Labeling for Meetings. Read-only. Verify your licensing/tenant: the Meetings scope and Teams-Premium meeting protections are evolving; the ContentType wording for the Meetings scope may differ in your tenant — confirm in the Purview portal. > [CmdletBinding()] param([string]$MeetingScopeMatch = 'Meeting' # substring matched against label ContentType/scope, verify per tenant)

How to run: .\IPMEET-Discover.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession (Security & Compliance PowerShell, for Get-Label / Get-LabelPolicy) and Connect-MicrosoftTeams (for Get-CsTeamsMeetingPolicy). Sign in interactively with a modern-auth/MFA admin; no secret is stored in

the script. EXPECTED RESULTS A summary: labels total and how many are Meetings-scoped (listed), the label policies present, and the Teams meeting-policy baseline — each as a PASS/INFO line to record against the Requirements checklist. .NOTES Purview — Sensitivity Labeling for Meetings. Read-only. Verify your licensing/tenant: the Meetings scope and Teams-Premium meeting protections are evolving; the ContentType wording for the Meetings scope may differ in your tenant — confirm in the Purview portal. > [CmdletBinding()] param([string]$MeetingScopeMatch = 'Meeting' # substring matched against label ContentType/scope, verify per tenant)

IPMEET-Report.ps1

Purpose: Builds 'Meeting Labels - Coverage Report' — for each meeting sensitivity label: its display name, scope (whether Meetings is in ContentType), the label policy that publishes it, and the policy's target coverage — as the management evidence artifact. Read-only; writing a CSV is opt-in. HOW TO RUN .\IPMEET-Report.ps1 # console table only (no file written) .\IPMEET-Report.ps1 -CsvPath .\MeetingLabelsReport.csv # also export to CSV PowerShell 5.1+/7+. Connect first with Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A table (and optional CSV) listing each meeting label, its Meetings scope, publishing policy, and target coverage — re-runnable on a schedule to keep coverage healthy. Makes no configuration change. .NOTES Purview — Sensitivity Labeling for Meetings. Read-only reporting. Verify your licensing/tenant: policy label-attachment and target property names vary by ExchangeOnlineManagement/S&C module version. > [CmdletBinding()] param([string]$MeetingScopeMatch = 'Meeting', [string]$CsvPath)

How to run: .\IPMEET-Report.ps1 # console table only (no file written) .\IPMEET-Report.ps1 -CsvPath .\MeetingLabelsReport.csv # also export to CSV PowerShell 5.1+/7+. Connect first with Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A table (and optional CSV) listing each meeting label, its Meetings scope, publishing policy, and target coverage — re-runnable on a schedule to keep coverage healthy. Makes no configuration change. .NOTES Purview — Sensitivity Labeling for Meetings. Read-only reporting. Verify your licensing/tenant: policy label-attachment and target property names vary by ExchangeOnlineManagement/S&C module version. > [CmdletBinding()] param([string]$MeetingScopeMatch = 'Meeting', [string]$CsvPath)

IPMEET-Set-MeetingLabel.ps1

Purpose: Automates the safe, repeatable LIFECYCLE of the meeting labels and their policy: creates the three meeting label objects (Meeting - General / Confidential / Highly Confidential) and publishes / points the 'Meeting Labels - Policy' at the pilot (or expansion) users. DEFAULTS TO -WhatIf: it prints exactly what it would do plus the reverse action and changes nothing; it applies only with -Commit. The per-meeting PROTECTION settings (lobby / recording / chat / copy / watermark / E2EE) are completed on each label in the Purview portal — this script does not assert them, because they are portal-configured and Teams-Premium-gated. No credentials or secrets are stored in the script. HOW TO RUN .\IPMEET-Set-MeetingLabel.ps1 -Action CreateLabels -WhatIf .\IPMEET-Set-MeetingLabel.ps1 -Action CreateLabels -Commit .\IPMEET-Set-MeetingLabel.ps1 -Action PublishPolicy -PolicyMembers "MeetingLabels-Pilot@contoso.com" -WhatIf .\IPMEET-Set-MeetingLabel.ps1 -Action PublishPolicy -PolicyMembers "MeetingLabels-Pilot@contoso.com" -Commit PowerShell 5.1+/7+. Connect first with Connect-IPPSSession (modern-auth/MFA admin). EXPECTED RESULTS -WhatIf: a line per label/policy stating what WOULD be created/published, plus the reverse command; no change is made. -Commit: the label objects and/or the policy exist afterward (verify with Get-Label / Get-LabelPolicy); finish the meeting-protection settings in the portal. .NOTES Purview — Sensitivity Labeling for Meetings. Safe-by-default. Verify your licensing/tenant: the Meetings scope + meeting protections are

portal-configured and Teams-Premium-gated; author them in the Purview portal after the label objects exist. Pilot before enforcing. > [CmdletBinding()] param([ValidateSet('CreateLabels','PublishPolicy')] [string]$Action = 'CreateLabels', [string]$PolicyName = 'Meeting Labels - Policy', [string[]]$PolicyMembers = @(), # UPNs / mail-enabled groups to publish the policy to [switch]$WhatIf, [switch]$Commit)

How to run: .\IPMEET-Set-MeetingLabel.ps1 -Action CreateLabels -WhatIf .\IPMEET-Set-MeetingLabel.ps1 -Action CreateLabels -Commit .\IPMEET-Set-MeetingLabel.ps1 -Action PublishPolicy -PolicyMembers "MeetingLabels-Pilot@contoso.com" -WhatIf .\IPMEET-Set-MeetingLabel.ps1 -Action PublishPolicy -PolicyMembers "MeetingLabels-Pilot@contoso.com" -Commit PowerShell 5.1+/7+. Connect first with Connect-IPPSSession (modern-auth/MFA admin). EXPECTED RESULTS -WhatIf: a line per label/policy stating what WOULD be created/published, plus the reverse command; no change is made. -Commit: the label objects and/or the policy exist afterward (verify with Get-Label / Get-LabelPolicy); finish the meeting-protection settings in the portal. .NOTES Purview — Sensitivity Labeling for Meetings. Safe-by-default. Verify your licensing/tenant: the Meetings scope + meeting protections are portal-configured and Teams-Premium-gated; author them in the Purview portal after the label objects exist. Pilot before enforcing. > [CmdletBinding()] param([ValidateSet('CreateLabels','PublishPolicy')] [string]$Action = 'CreateLabels', [string]$PolicyName = 'Meeting Labels - Policy', [string[]]$PolicyMembers = @(), # UPNs / mail-enabled groups to publish the policy to [switch]$WhatIf, [switch]$Commit)

IPMEET-Validate.ps1

Purpose: The pilot/expansion gate. Confirms PASS/FAIL that each intended meeting label (Meeting - General / Confidential / Highly Confidential) exists, is scoped to Meetings, and is published in the 'Meeting Labels - Policy', and that the Teams meeting-policy baseline is readable/compatible. Read-only — makes no change. Run on the pilot ring before expanding, and after each expansion wave. HOW TO RUN .\IPMEET-Validate.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession and (optionally) Connect-MicrosoftTeams. No secret is stored in the script. EXPECTED RESULTS A PASS/FAIL line per gate; a final overall PASS only when every intended label exists, is Meetings-scoped, and is published in the policy. Any FAIL names the specific missing object. .NOTES Purview — Sensitivity Labeling for Meetings. Read-only gate. Verify your licensing/tenant: confirm the Meetings scope wording in ContentType and the policy parameter names for your module version. > [CmdletBinding()] param([string]$PolicyName = 'Meeting Labels - Policy', [string[]]$ExpectedLabels = @('Meeting - General','Meeting - Confidential','Meeting - Highly Confidential'), [string]$MeetingScopeMatch = 'Meeting')

How to run: .\IPMEET-Validate.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession and (optionally) Connect-MicrosoftTeams. No secret is stored in the script. EXPECTED RESULTS A PASS/FAIL line per gate; a final overall PASS only when every intended label exists, is Meetings-scoped, and is published in the policy. Any FAIL names the specific missing object. .NOTES Purview — Sensitivity Labeling for Meetings. Read-only gate. Verify your licensing/tenant: confirm the Meetings scope wording in ContentType and the policy parameter names for your module version. > [CmdletBinding()] param([string]$PolicyName = 'Meeting Labels - Policy', [string[]]$ExpectedLabels = @('Meeting - General','Meeting - Confidential','Meeting - Highly Confidential'), [string]$MeetingScopeMatch = 'Meeting')